

Machine Learning Regression Models: Deep Revision Notes

1. Linear Regression

Core Idea (Intuition First)

- **Problem:** Find the best linear relationship between features and a continuous target
- **Why it exists:** Simplest parametric model that allows interpretability and closed-form solution
- **Core assumption:** The true relationship is linear (or can be approximated linearly)
- **Trade-off:** Sacrifices flexibility for interpretability and computational efficiency

Mathematical Formulation

Model equation:

$$\hat{y} = X\beta + \beta_0$$

or vectorized: $\hat{y} = X\beta$ (with intercept absorbed)

Loss function (RSS - Residual Sum of Squares):

$$L(\beta) = \sum (y_i - \hat{y}_i)^2 = \|y - X\beta\|^2$$

What's being optimized:

- Minimize squared vertical distance between predictions and true values
- Closed-form solution: $\beta = (X^T X)^{-1} X^T y$

Why squared error?

- Mathematically tractable (differentiable, convex)
- Penalizes large errors more (outlier sensitive)
- Corresponds to MLE under Gaussian noise assumption

Assumptions (Very Important)

1. **Linearity:** True relationship is linear

- *Violated* → Systematic bias, poor predictions, underfit model
- *Check via*: residual plots (should show no pattern)

2. **Independence**: Observations are independent

- *Violated* → Standard errors underestimated, overconfident predictions
- *Common in*: time series, clustered data

3. **Homoscedasticity**: Constant variance of errors ($\epsilon \sim N(0, \sigma^2)$)

- *Violated* → Inefficient estimates, invalid confidence intervals
- *Check via*: residuals vs fitted plot (should show constant spread)

4. **Normality of errors**: $\epsilon \sim N(0, \sigma^2)$

- *Violated* → Confidence intervals unreliable (less critical for large n)
- *Note*: Not needed for unbiased estimates, only for inference

5. **No multicollinearity**: Features are not highly correlated

- *Violated* → Unstable coefficients, high variance, $(X^T X)^{-1}$ ill-conditioned
- *Check via*: VIF (Variance Inflation Factor) > 10 is problematic

Bias–Variance Perspective

- **Position**: Low bias, high variance (when p is large relative to n)
- **Why high variance**: Fits noise in training data, coefficients swing wildly with small data changes
- **When $n \gg p$** : Variance decreases, becomes more stable
- **Problem**: With many features or correlated features, variance explodes

High Variance ←———— [Linear Reg] —————→ Low Variance
 Low Bias High Bias

Optimization & Learning Process

Closed-form solution:

- $\beta = (X^T X)^{-1} X^T y$
- No learning rate needed
- **Problem**: $O(p^3)$ complexity, breaks when $X^T X$ is singular (multicollinearity)

Gradient Descent (when n or p is large):

$$\beta \leftarrow \beta - \alpha \cdot \nabla L(\beta)$$

where $\nabla L(\beta) = -2X^T(y - X\beta)$

Effect of learning rate α :

- Too large $\rightarrow$ oscillation, divergence
- Too small $\rightarrow$ slow convergence
- **Convergence:** Guaranteed if $\alpha < 2/\lambda_{\max}(X^T X)$

Convergence behavior:

- Convex loss $\rightarrow$ guaranteed global minimum
- Linear convergence rate (error decreases exponentially)

Evaluation Metrics & Interpretation

MSE (Mean Squared Error): $\frac{\sum (y_i - \hat{y}_i)^2}{n}$

- Same units as y^2 , sensitive to outliers
- **Misleading when:** Outliers present, or when you care about absolute error

RMSE (Root Mean Squared Error): $\sqrt{\text{MSE}}$

- Same units as y , interpretable scale
- **Misleading when:** Distribution has heavy tails

MAE (Mean Absolute Error): $\frac{\sum |y_i - \hat{y}_i|}{n}$

- Robust to outliers, same units as y
- **Misleading when:** Large errors should be penalized more

R² (Coefficient of Determination): $1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$

- Fraction of variance explained (0 to 1, higher is better)
- **Misleading when:**
 - Adding features (always increases, use adjusted R²)
 - Non-linear relationships
 - Extrapolation (can be negative on test set)

Common Failure Modes & Mistakes

1. Multicollinearity

- Symptom: Large coefficients with opposite signs, high variance
- Detection: $VIF > 10$, condition number of $X^T X > 30$
- Fix: Remove correlated features, use Ridge/Lasso, PCA

2. Extrapolation

- Linear model fails catastrophically outside training range
- Always check feature ranges between train/test

3. Outliers

- Squared loss heavily influenced by outliers
- Single point can dominate the fit
- Fix: Robust regression (Huber loss), outlier detection

4. Feature scaling ignored

- Affects gradient descent convergence
- Doesn't affect closed-form solution, but affects interpretation

5. Misinterpreting coefficients

- β_j is effect of x_j *holding all other features constant*
- With multicollinearity, coefficients are meaningless individually
- Correlation $\neq$ causation (always)

6. Using R^2 on test set

- Can be negative if model is terrible
- Doesn't indicate generalization

Practical Use Cases

When to use:

- Need interpretable model (feature importance clear)
- Linear relationships are reasonable approximation
- Quick baseline before complex models

- $n \gg p$ (many observations, few features)
- Predictions are more important than perfect fit

When NOT to use:

- Non-linear relationships (use polynomial features or non-linear models)
 - $p \approx n$ or $p > n$ (use Ridge/Lasso)
 - Multicollinearity present (use Ridge)
 - Outliers dominate (use robust regression)
 - Binary/categorical outcome (use Logistic)
-

2. Ridge Regression (L2 Regularization)

Core Idea (Intuition First)

- **Problem:** Linear regression has high variance when features are correlated or p is large
- **Solution:** Penalize large coefficients to reduce model complexity
- **Key insight:** Bias the coefficients toward zero in exchange for lower variance
- **Trade-off:** Accept some bias to dramatically reduce variance

Mathematical Formulation

Model equation:

$$\hat{y} = X\beta \text{ (same as linear regression)}$$

Loss function (Penalized RSS):

$$\begin{aligned} L(\beta) &= \|y - X\beta\|^2 + \lambda \|\beta\|^2 \\ &= \text{RSS} + \lambda \sum \beta_j^2 \end{aligned}$$

What's being optimized:

- Minimize prediction error AND coefficient magnitude
- λ controls bias-variance trade-off
- Closed-form solution: $\beta = (X^T X + \lambda I)^{-1} X^T y$

Key observation:

- λI added to $X^T X$ makes it invertible even with multicollinearity
- $\lambda \rightarrow 0$: Ridge becomes Linear Regression
- $\lambda \rightarrow \infty$: $\beta \rightarrow 0$ (predicts mean)

Assumptions (Very Important)

- Same as Linear Regression (linearity, independence, homoscedasticity)
- **Key difference:** Robust to multicollinearity (doesn't assume features are uncorrelated)
- **New assumption:** Features should be standardized (penalty is scale-dependent)

What happens if not standardized:

- Features with large scales get penalized less
- λ has different effective strength per feature
- **Always standardize** before Ridge

Bias–Variance Perspective

- **Position:** Trades increased bias for decreased variance
- **Effect of λ :**
 - $\lambda = 0$: Same as Linear Regression (high variance)
 - λ increases: Bias increases, variance decreases
 - $\lambda \rightarrow \infty$: Maximum bias (predicts mean), zero variance

Linear Reg	Ridge (λ small)	Ridge (λ large)
High Variance	Balanced	High Bias
Low Bias	Moderate	Low Variance

Bias-Variance decomposition:

$$E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

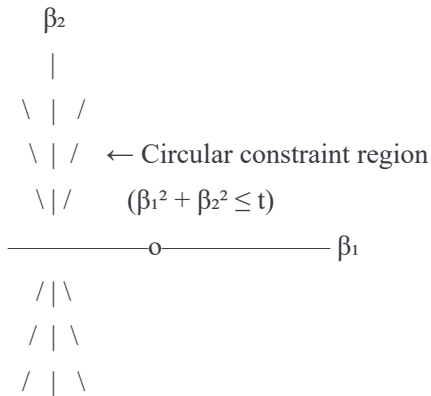
Ridge: $\uparrow$ Bias, $\downarrow$ Variance (net win when variance was high)

Role of Regularization

L2 penalty ($\|\beta\|^2$):

- Shrinks all coefficients toward zero proportionally
- **Never sets coefficients to exactly zero**
- Smooth, differentiable everywhere

Geometric intuition:



- RSS ellipses intersect smoothly with circle
- Solutions lie inside circle (shrinkage)

Effect on coefficients:

- Coefficients shrink by factor $(X^T X + \lambda I)^{-1}$ compared to OLS
- Correlated features have similar magnitudes (distributed penalty)
- Stabilizes estimates when features are correlated

Optimization & Learning Process

Closed-form solution:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

- Always invertible (λI ensures positive definite)
- Computationally stable

Gradient Descent:

$$\beta \leftarrow \beta - \alpha \cdot \nabla L(\beta)$$

where $\nabla L(\beta) = -2X^T(y - X\beta) + 2\lambda\beta$

Effect of λ :

- Acts as learning rate modifier
- Larger $\lambda \rightarrow$ faster convergence (stronger pull toward zero)
- Must tune via cross-validation

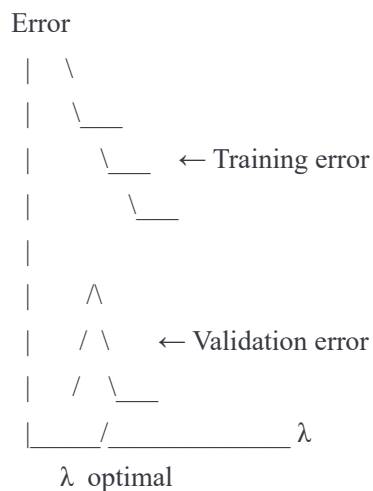
Convergence:

- Still convex, guaranteed global minimum
- Condition number improved: $\kappa(X^T X + \lambda I) < \kappa(X^T X)$

Evaluation Metrics & Interpretation

- Same metrics as Linear Regression (MSE, RMSE, MAE, R^2)
- **Critical:** Use cross-validation for λ selection
- Plot validation error vs λ to find optimal regularization

λ selection:



Coefficient interpretation:

- Coefficients are biased (shrunk toward zero)
- Relative magnitudes still meaningful
- Cannot interpret as "unit change in $x \rightarrow \beta$ change in y " (not unbiased)

Common Failure Modes & Mistakes

1. Forgetting to standardize features

- Features with large scales dominate
- λ has inconsistent effect
- **Always:** `StandardScaler().fit_transform(X)`

2. λ too large (over-regularization)

- Underfitting, high bias
- All coefficients near zero
- Check: training error high

3. λ too small (under-regularization)

- Doesn't solve multicollinearity
- Still high variance
- Check: validation error much worse than training

4. Using test set for λ tuning

- Data leakage
- Overly optimistic performance
- **Always:** nested cross-validation

5. Not standardizing y

- For Ridge, only standardize X
- Standardizing y changes interpretation of λ

6. Comparing coefficients across λ values

- Coefficients scale with λ
- Use model performance, not coefficient magnitude

Practical Use Cases

When to use:

- Multicollinearity present (features correlated)
- $p \approx n$ (many features relative to samples)

- All features are potentially relevant (want to keep all)
- Need stable, interpretable coefficients
- Prediction accuracy > feature selection

When NOT to use:

- Sparse true model (many irrelevant features) → Use Lasso
- Need exact feature selection → Use Lasso
- No multicollinearity and $n \gg p$ → Use Linear Regression
- Non-linear relationships → Use kernel Ridge or non-linear models

Ridge vs Linear Regression:

- Ridge almost always better in practice (bias-variance trade-off)
 - Only exception: perfect conditions for Linear Regression ($n \gg p$, no multicollinearity)
-

3. Lasso Regression (L1 Regularization)

Core Idea (Intuition First)

- **Problem:** Ridge doesn't do feature selection (keeps all features)
- **Solution:** Use L1 penalty that can set coefficients to exactly zero
- **Key insight:** L1 norm is non-differentiable at zero, allowing sparse solutions
- **Trade-off:** Feature selection + regularization, but unstable with correlated features

Mathematical Formulation

Model equation:

$$\hat{y} = X\beta \text{ (same as linear regression)}$$

Loss function:

$$\begin{aligned} L(\beta) &= \|y - X\beta\|^2 + \lambda \|\beta\|_1 \\ &= \text{RSS} + \lambda \sum |\beta_j| \end{aligned}$$

What's being optimized:

- Minimize prediction error AND number of non-zero coefficients
- λ controls sparsity (higher $\lambda \rightarrow$ more zeros)
- **No closed-form solution** (L1 norm not differentiable at 0)
- Solved via coordinate descent, LARS, or proximal gradient

Key observation:

- As λ increases, coefficients hit zero one by one
- Path of solutions is piecewise linear (useful for regularization path plots)

Assumptions (Very Important)

- Same base assumptions as Linear Regression
- **Critical assumption:** Features should be standardized (scale-dependent)
- **Instability with correlation:** If $x_1 \approx x_2$, Lasso arbitrarily picks one

What happens with correlated features:

- Lasso picks one arbitrarily, sets others to zero
- Solution unstable (small data changes $\rightarrow$ different feature selected)
- **Fix:** Use Elastic Net ($\alpha \cdot L1 + (1-\alpha) \cdot L2$)

Bias–Variance Perspective

- **Position:** Similar to Ridge, but with sparse solutions
- **Trade-off:**
 - High bias (sets coefficients to exactly zero)
 - Lower variance (fewer parameters)
 - More bias than Ridge for same λ (harsher penalty)

Linear Reg	Lasso (λ small)	Lasso (λ large)
High Variance	Sparse solution	Very sparse
Low Bias	Moderate bias	High bias

Comparison with Ridge:

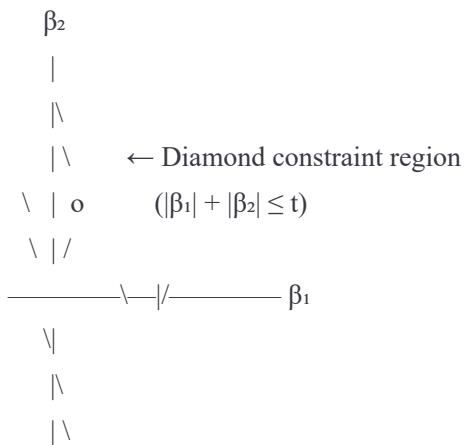
- Lasso: Higher bias, potentially lower variance (if true model is sparse)
- Ridge: Lower bias, higher variance (keeps all features)

Role of Regularization

L1 penalty ($\|\beta\|_1$):

- Pushes coefficients to exactly zero
- Automatic feature selection
- Non-differentiable at zero (creates sparsity)

Geometric intuition:



- RSS ellipses intersect at corners (sparse solutions)
- Corners correspond to some $\beta_j = 0$

Effect on coefficients:

- Small coefficients $\rightarrow 0$ (hard thresholding)
- Large coefficients $\rightarrow$ shrunk but not to zero
- Correlated features: one chosen, others zeroed

L1 vs L2 intuition:

Aspect	L1 (Lasso)	L2 (Ridge)
Penalty shape	Diamond (corners)	Circle (smooth)
Sparsity	Yes (feature selection)	No (all features kept)
Stability	Unstable with correlation	Stable
Interpretability	Simpler (fewer features)	Complex (all features)

Optimization & Learning Process

No closed-form solution (L1 norm not differentiable)

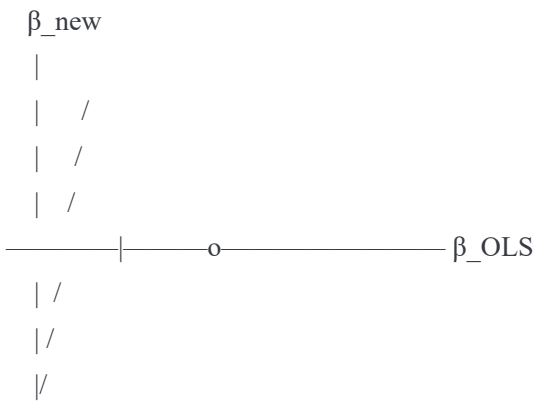
Coordinate Descent (most common):

For each β_j :

1. Fix all other coefficients
2. Minimize $L(\beta_j)$ (1D optimization)
3. Apply soft-thresholding:

$$\beta_j \leftarrow \text{sign}(\beta_{j_OLS}) \cdot \max(0, |\beta_{j_OLS}| - \lambda/2)$$

Soft-thresholding operator:



- Sets coefficients to 0 if $|\beta_{OLS}| < \lambda/2$
- Otherwise shrinks by $\lambda/2$

Convergence:

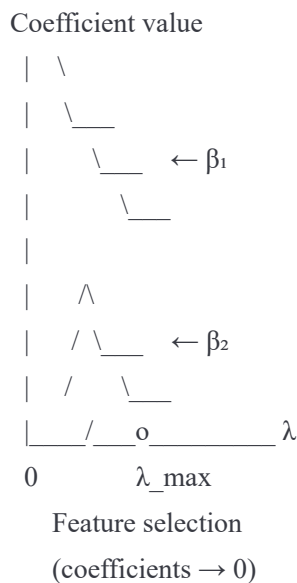
- Coordinate descent converges to global minimum (convex)
- Faster than gradient descent for Lasso

- Regularization path can be computed efficiently (LARS algorithm)

Evaluation Metrics & Interpretation

- Same metrics as Linear/Ridge Regression
- **Additional:** Number of non-zero coefficients (model complexity)
- **λ selection:** Cross-validation, plot validation error vs λ

Regularization path plot:



Coefficient interpretation:

- Non-zero coefficients: selected features
- Zero coefficients: deemed irrelevant
- **Caution:** With correlated features, selection is arbitrary

Common Failure Modes & Mistakes

1. Correlated features instability

- Lasso picks one arbitrarily, ignores others
- Solution changes with small data perturbations
- **Fix:** Elastic Net or grouped Lasso

2. Over-regularization (λ too large)

- All coefficients $\rightarrow 0$

- Model predicts mean
- Check: all coefficients zero, high training error

3. Under-regularization (λ too small)

- No feature selection (all features kept)
- Equivalent to Linear Regression
- Check: all features have non-zero coefficients

4. Forgetting to standardize

- Features with large scales less likely to be zeroed
- λ has inconsistent effect
- **Always:** standardize features

5. Trusting feature selection blindly

- Selected features $\neq$ causal features
- Omitted variables can be important
- Use domain knowledge + Lasso

6. Using selected features in Linear Regression

- "Post-selection inference" problem
- Standard errors invalid (selection process not accounted for)
- Use Lasso coefficients directly or bootstrap

Practical Use Cases

When to use:

- Sparse true model (many irrelevant features)
- Need interpretability + feature selection
- $p \gg n$ (more features than samples)
- Features are weakly correlated
- Exploratory analysis (identify relevant features)

When NOT to use:

- Features are highly correlated (unstable) $\rightarrow$ Use Ridge or Elastic Net
- All features are relevant $\rightarrow$ Use Ridge

- Need stable feature selection → Use stability selection or Elastic Net
- Groups of correlated features (want to select together) → Use Grouped Lasso

Lasso vs Ridge:

- **Lasso:** Feature selection, sparser, less stable
- **Ridge:** No feature selection, keeps all, more stable
- **Elastic Net:** Best of both ($\alpha \cdot L1 + (1-\alpha) \cdot L2$)

Golden rule:

- Many weak features → Ridge
 - Few strong features → Lasso
 - Unsure → Elastic Net
-

4. Logistic Regression

Core Idea (Intuition First)

- **Problem:** Linear regression outputs unbounded values, but we need probabilities $[0, 1]$ for classification
- **Solution:** Model log-odds (logit) as linear function, then apply sigmoid
- **Key insight:** Separate classes by finding linear decision boundary in feature space
- **Why "regression":** We're regressing on log-odds, not class labels directly

Fundamental difference from Linear Regression:

- Linear: Models $E[y|X]$ directly (unbounded)
- Logistic: Models $P(y=1|X)$ via link function (bounded $[0, 1]$)

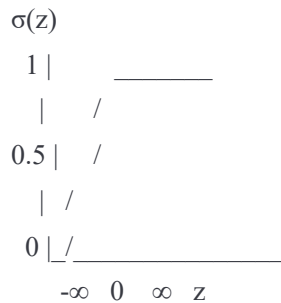
Mathematical Formulation

Model equation:

$$z = X\beta \text{ (linear combination)}$$

$$P(y=1|X) = \sigma(z) = 1 / (1 + e^{(-z)})$$

Sigmoid function:



Properties:

- $\sigma(0) = 0.5$ (decision boundary)
- $\sigma(\infty) \rightarrow 1, \sigma(-\infty) \rightarrow 0$
- Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Loss function (Binary Cross-Entropy / Log Loss):

$$L(\beta) = -\sum [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Why this loss?

- Derived from Maximum Likelihood Estimation (MLE)
- Convex (guaranteed global minimum)
- Penalizes confident wrong predictions heavily
- Differentiable (enables gradient descent)

What's being optimized:

- Maximize likelihood of observed class labels
- Equivalently, minimize cross-entropy
- **No closed-form solution** (requires iterative optimization)

Assumptions (Very Important)

1. **Linearity of log-odds:** $\log(p/(1-p))$ is linear in X
 - *Violated* $\rightarrow$ Non-linear decision boundary needed
 - *Check via:* Plot predicted probabilities vs features
2. **Independence:** Observations are independent

- *Violated* → Standard errors underestimated
 - *Common in*: Time series, hierarchical data
3. **No perfect multicollinearity**: Features not perfectly correlated
- *Violated* → Unstable coefficients, convergence issues
 - *Check via*: VIF, condition number
4. **Large sample size**: Need enough samples per feature
- *Rule of thumb*: At least 10 events per predictor (EPV = 10)
 - *Violated* → Overfitting, unstable estimates
5. **Binary outcome**: $y \in \{0, 1\}$
- For multi-class: Use multinomial/softmax logistic regression

Key difference from Linear Regression:

- No assumption of normality of errors
- No assumption of homoscedasticity
- Outcome must be binary (or counts for Poisson variant)

Bias–Variance Perspective

- **Position**: Similar to Linear Regression
 - Low bias (flexible linear boundary)
 - High variance (when p is large or n is small)

With regularization (L1/L2):

- Ridge Logistic: Stable with correlated features
- Lasso Logistic: Feature selection
- Trade-off same as regression case

High Variance ← [Logistic Reg] → Low Variance
 Low Bias (no reg) High Bias
 (with reg)

Optimization & Learning Process

Gradient Descent:

$$\beta \leftarrow \beta - \alpha \cdot \nabla L(\beta)$$

where $\nabla L(\beta) = X^T(\hat{y} - y)$

Note: Same form as Linear Regression, but $\hat{y} = \sigma(X\beta)$ instead of $X\beta$

Learning rate effects:

- Too large $\rightarrow$ oscillation around minimum
- Too small $\rightarrow$ slow convergence
- Adaptive methods (Adam, RMSprop) work well

Convergence:

- Loss is convex $\rightarrow$ guaranteed global minimum
- Typically converges in 100-1000 iterations
- Monitor: loss decrease, gradient magnitude

Newton's method (second-order):

- Uses Hessian matrix (second derivatives)
- Faster convergence (quadratic vs linear)
- Computationally expensive for large p

Evaluation Metrics & Interpretation

Confusion Matrix:

		Predicted	
		0	1
Actual	0	TN	FP
	1	FN	TP

Accuracy: $(TP + TN) / (TP + TN + FP + FN)$

- **Misleading when:** Class imbalance (e.g., 95% class 0)

Precision: $TP / (TP + FP)$

- "Of all predicted positives, how many are correct?"
- **Use when:** False positives are costly (spam detection)

Recall (Sensitivity): $TP / (TP + FN)$

- "Of all actual positives, how many did we catch?"
- **Use when:** False negatives are costly (disease detection)

F1-Score: $2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall})$

- Harmonic mean of precision and recall
- **Use when:** Need balance, class imbalance present

ROC-AUC (Receiver Operating Characteristic - Area Under Curve):

TPR (Recall)

1 | /

| /

| / ____ ← Random classifier (AUC = 0.5)

| /

0 | _____

0 1

FPR (1-Specificity)

- AUC = 1: Perfect classifier
- AUC = 0.5: Random classifier
- AUC < 0.5: Worse than random (flip predictions)
- **Advantage:** Threshold-independent
- **Use when:** Want overall performance across all thresholds

Log-Loss (Cross-Entropy):

- Measures quality of probability predictions
- Lower is better
- **Use when:** Care about calibrated probabilities, not just class

Calibration:

- Are predicted probabilities accurate?
- Plot: Predicted probability vs actual frequency
- **Check:** Calibration plot should be diagonal

Common Failure Modes & Mistakes

1. Class imbalance

- Symptom: Model predicts majority class always
- Accuracy looks good, but useless
- **Fix:**
 - Resample (oversample minority, undersample majority)
 - Class weights (penalize minority class errors more)
 - Use precision-recall instead of ROC

2. Multicollinearity

- Coefficients unstable, large standard errors
- VIF > 10 problematic
- **Fix:** Ridge regularization, remove correlated features

3. Separation (perfect or quasi-)

- When a feature perfectly separates classes
- Coefficients $\rightarrow \pm\infty$, warnings during training
- **Fix:** Regularization (Ridge/Lasso), remove perfectly separating feature

4. Threshold selection

- Default threshold = 0.5 often suboptimal
- With imbalanced classes, adjust threshold based on cost
- **Fix:** Use ROC curve to select optimal threshold

5. Ignoring calibration

- Predicted probabilities not meaningful
- Model says 70% but it's actually 90%
- **Fix:** Calibration methods (Platt scaling, isotonic regression)

6. Misinterpreting coefficients

- β_j is effect on log-odds, not probability
- $\exp(\beta_j)$ = odds ratio (multiplicative effect)
- With correlated features, coefficients misleading

7. Data leakage

- Features that contain information from future
- Using entire dataset for scaling before split
- **Fix:** Fit scaler on training set only

8. Extrapolation

- Sigmoid saturates at extremes
- Predictions become insensitive to feature changes
- Check: Feature ranges in train vs test

Practical Use Cases

When to use:

- Binary classification with linear decision boundary
- Need interpretable model (coefficients $\rightarrow$ odds ratios)
- Need probability estimates (not just class labels)
- Baseline for more complex classifiers
- Small to medium-sized datasets
- Online learning (update with new data easily)

When NOT to use:

- Non-linear decision boundary (use kernel methods, tree ensembles, neural nets)
- Multi-class with complex relationships (use softmax, neural nets)
- Extremely imbalanced classes (consider anomaly detection methods)
- Image/text classification (use deep learning)
- Perfect separation exists (use regularization or different model)

Logistic vs Linear Regression:

- Logistic: Binary outcome, outputs probabilities, uses sigmoid
- Linear: Continuous outcome, outputs real numbers, no transform

Real-world examples:

- Email spam detection (spam vs not spam)
- Credit default prediction (default vs no default)
- Medical diagnosis (disease vs healthy)
- Click-through rate prediction (click vs no click)
- Customer churn (churn vs retain)

Comparative Summary Table

Aspect	Linear Regression	Ridge Regression	Lasso Regression	Logistic Regression
Objective	Predict continuous outcome	Predict continuous outcome with stability	Predict continuous outcome + feature selection	Predict binary outcome (probabilities)
Loss Function	RSS: $\sum (y_i - \hat{y}_i)^2$	$RSS + \lambda \sum \beta_j^2$	$RSS + \lambda \sum \beta_j $	$-\sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
Regularization	None	L2 (shrinks coefficients toward 0)	L1 (sets coefficients to exactly 0)	Optional (Ridge/Lasso/Elastic Net)
Closed-Form Solution	Yes: $\beta = (X^T X)^{-1} X^T y$	Yes: $\beta = (X^T X + \lambda I)^{-1} X^T y$	No (coordinate descent)	No (gradient descent)
Feature Selection	No	No (keeps all features)	Yes (automatic)	No (unless L1 regularized)
Interpretability	High (coefficients = effect size)	Moderate (biased coefficients)	High (only relevant features)	High ($\exp(\beta)$ = odds ratio)
Bias-Variance	Low bias, high variance	Moderate bias, lower variance	Moderate bias, low variance (if sparse)	Low bias, high variance (no reg)

Aspect	Linear Regression	Ridge Regression	Lasso Regression	Logistic Regression
Handles Multicollinearity	No ($X^T X$ singular)	Yes (stable coefficients)	Picks one feature, drops others	No (coefficients unstable)
Best When	$n \gg p$, no correlation, linear relationship	Correlated features, $p \approx n$, need all features	Sparse model, feature selection needed	Binary outcome, linear log-odds
Typical Failure Cases	Multicollinearity, outliers, non-linearity, extrapolation	Over/under-regularization, forgetting to standardize	Correlated features (unstable), trusting selection blindly	Class imbalance, separation, non-linear boundary, misc